

Finance AI Playbook

财务部门的 AI 转型实战手册

刘晃 (Alex Liu)

一个财务人从真实场景出发，写给财务团队的 AI 转型实战 Playbook。

目录

引 关于这本手册	3
1 第一章：财务 AI 的三条路	4
2 第二章：把重复变成基础设施	9
3 第三章：AI 提问，人澄清，数据说话	16
4 第四章：自动化之前，先让流程可见	22
5 第五章：财务 AI 的红线	27
6 第六章：三个真实场景	32

关于这本手册

作者：刘晃（Alex Liu）

2026年

这不是一份技术手册，也不是一份方法论清单。

它是一个财务人走过这条路之后，把走过的路记下来的东西。

有些地方还在摸索，有些工具还在迭代，有些判断可能在六个月后需要更新。但这里写的每一件事，都是真实发生过的——真实的场景，真实的决策，真实的坑。

希望它能成为你的垫脚石。

第一章：财务 AI 的三条路

读完这章，你会知道为什么大多数财务部门卡在原地，以及真正的转型从哪里开始。

一个让我印象深刻的失败

有一段时间，我非常认真地相信“AI-native 工作流”这个概念。

那时候，这个词开始在各种场合出现——会议上有人提，文章里有人写，大家都在说财务部门即将被彻底颠覆，一切重复性工作都会消失，人只需要做判断和决策。

听起来很对。我想知道怎么做到。

所以我去问 AI。

我把我们财务部门的情况描述给它，问它：如果要实现真正的 AI-native 工作流，我应该从哪里开始？

AI 给了我一个很好看的框架。有层次，有逻辑，读起来头头是道。

但当我试图把那个框架落地的时候，我发现：它没有办法告诉我，在我们这个具体的系统、这个具体的数据架构、这个具体的团队里，第一步应该做什么。

概念是对的，路径是空的。

那一刻我意识到：这条路，没有人能替我走。

三条路，大多数人在第一条

在我开始摸索之后，我观察到财务部门在 AI 转型上，走的其实是三条路。

不是三个阶段，是三种选择——因为有些人一直留在第一条路上，从来没有走到第二条和第三条。

第一条路：修补

在现有的工作方式上，加一点 AI。

写报告之前，先让 AI 起个草。整理会议记录的时候，让 AI 做个摘要。分析一段数据，让 AI 帮忙解读。

这没有什么不对。这些事情 AI 确实能做，而且能帮你省时间。

但修补有一个根本性的局限：你的工作流没有变，AI 只是在原来的工作流里做了一些替换。

效率提升是真实的，但有天花板。因为你提升的，只是流程里的某几个点，而不是整个流程的设计逻辑。

大多数财务部门，现在处于这个阶段。用了 AI，但用法是：有问题去问一下，有任务让它帮一下，然后继续按原来的方式工作。

第二条路：颠覆

有一段时间，我想过彻底不一样的方式。

如果不是在现有流程里加 AI，而是从零开始重新设计——假设 AI 已经无所不能，财务部门应该是什么样子？

这个方向很诱人，也确实打开了一些思路。

但我很快发现了它的问题：现实不允许你从零开始。

系统还在跑，数据还要出，报告还要交。你没有办法暂停整个财务部门的运作，等你把“理想中的 AI-native 工作流”设计完再重启。

颠覆式的想象，有时候是必要的——它帮你看清楚方向。但它不是一个可以直接执行的路径。

第三条路：重塑

重塑，不是修补，也不是颠覆。

它是这样一种方式：在现有系统能运转的前提下，一步一步地把工作流的底层逻辑改掉。

不是替换某几个动作，而是改变“谁做什么、怎么触发、结果怎么流转”的整体设计。

重塑的节奏是慢的，但每一步都是真实的进展。而且每一步做完，它就稳定在那里——不会因为某个工具换了版本就失效，因为它已经变成了你工作流的基础设施。

这是我最终选择的路，也是这本 Playbook 试图描述的路。

为什么大多数财务部门卡在修补？

有几个真实的原因，不是借口，是障碍。

数据太敏感，不敢放。财务数据是公司最核心的机密，同时受到最严格的合规要求。第一反应是“不能随使用 AI 处理”，这个直觉是对的，但很多团队因此停在了“不知道什么能用”的状态，什么都不用。

系统太封闭，连不上。很多财务系统是十年前的架构，没有 API，没有批量导出，数据被锁在里面出不来。AI 再强，没有数据就没有办法工作。

流程太复杂，说不清。财务流程涉及的规则、例外、判断节点非常多。要让 AI 参与进来，首先要把流程说清楚——但很多流程是“靠经验判断，没有文档”的状态，说清楚本身就是一件难事。

没有人知道从哪里开始。AI-native 工作流的概念很好，但没有一个针对财务场景的、具体的、可以执行的起点。问 AI，它给你框架；问顾问，他们给你方法论；没有人告诉你，在你这个具体的系统和团队里，第一步做什么。

这四个障碍，Part III 接下来的章节会一一处理。

但在进入具体方法之前，有一件事需要先说清楚。

财务 AI 的特殊性：四个不能忽视的约束

财务场景有四个特质，使得它在 AI 应用上，需要比其他职能更谨慎、更系统。

约束一：数字有法律效力

财务数字不只是信息。它们出现在财务报表、税务申报、合同、监管报告里——一旦错了，后果不只是“数据不准”，而是审计风险、税务风险、法律责任。

这意味着：财务场景的 AI 输出验证要求，比任何其他职能都高一个量级。

约束二：强监管环境

会计准则、税法、上市公司信息披露要求、行业监管……财务工作在一个高度规则化的环境里运作。AI 的使用，必须在这个框架里发生，不能绕过它。

约束三：数据极度敏感

财务数据是公司最核心的机密之一。未经授权的数据流转，不只是合规问题，可能直接触发法律风险。

这决定了财务 AI 工具的选型，必须把数据安全放在第一位，而不是功能第一位。

约束四：审计可追溯

财务工作天然需要留痕。任何 AI 参与的环节，都必须能在审计时说得清楚：这个数字是怎么来的，经过了什么处理，谁做了最终确认。

这四个约束，不是阻止你用 AI 的理由，而是在告诉你：财务 AI 的设计，必须把这四件事内建进去，不能事后补丁。

这本 Playbook 的逻辑

接下来的章节，会按照这条线展开：

第二章 — 从识别重复开始，把手工操作变成可靠的自动化，再把自动化变成 AI 可以调用的基础设施。这是重塑的地基。

第三章 — 双入口设计：为什么同一套工具，需要同时支持人类点击和 Agent 调用。这是让不同成熟度的员工都能用起来的关键。

第四章 — AI 提问，人澄清，数据说话。财务分析里，人和 AI 各自负责什么，边界在哪里。

第五章 — 财务 AI 的红线。比通用版更严格的边界，以及如何在合规框架内使用 AI。

第六章 — 三个真实场景的完整流程。不是案例研究，是可以直接参考的操作路径。

☐ 本章要点卡

1. 财务 AI 转型有三条路：修补、颠覆、重塑——大多数人卡在修补，重塑是真正可持续的路径
 2. 卡住的原因不是能力问题，是四个真实障碍：数据敏感、系统封闭、流程说不清、没有起点
 3. 财务场景有四个特殊约束：数字法律效力、强监管、数据敏感、审计追溯——这四件事必须内建进设计，不能事后补
-

☐ 读完这章，你可以做的事

- [] 诚实评估你的团队：现在在三条路里的哪一条？是真的在重塑，还是以为自己在重塑，其实还在修补？
 - [] 列出你们财务部门目前最大的一个重复性手工操作——这会是第二章的起点
 - [] 对照四个约束，想想你们现在的 AI 使用方式，哪个约束目前处理得最不够？
-

下一章：把重复变成基础设施——从手工操作到 AI 可调用的工具

第二章：把重复变成基础设施

读完这章，你会明白如何把财务部门的手工重复操作，一步一步变成 AI 可以调用的标准化能力。

从一个真实的烦恼开始

财务部门有一类工作，每个人都做过，每个人都烦过：

定时去系统里导报表。

每个月固定时间，登进去，点几下，等它生成，下载，改文件名，放到对应的文件夹。如果管的主体多，这件事要做十几遍。有时候系统卡，有时候忘了，有时候导出来的格式不对还要手动处理。

这件事本身没有任何业务价值，但它每个月都必须发生，而且必须由人来做——或者说，过去必须由人来做。

这类工作，就是我们要改造的第一类目标。

识别"待封装的能力"

财务部门的重复操作，大概可以分三类：

第一类：从外部系统抓数据

税务局网站的发票信息、海关数据、监管报送平台的状态——这些数据存在于各种政府或第三方系统里，没有 API，只能人工登录操作。

这类任务的特点：步骤固定，判断少，频率高，但需要人守着。

第二类：从内部系统导报表

财务系统里的各类报表，按月、按季、按需导出。如果系统没有批量导出功能，或者无法直接连数据库，管理主体一多，就是大量重复点击。

这类任务的特点：完全可以描述清楚每一步，没有任何需要人判断的地方。

第三类：从数仓取数

如果已经有数据仓库，这类任务最接近标准化——本质上是把 SQL 查询封装好，按需触发，结果输出到指定位置。

这三类任务，有一个共同特征：它们现在消耗人的时间，但人在这个过程里没有贡献任何判断力。

这就是“待封装的能力”的识别标准——不需要人判断，只需要人执行的任务，都是待封装的能力。

第一步：用 RPA 固化重复操作

识别出目标之后，第一步是把这些操作固化成可以自动运行的脚本。

这就是 RPA (Robotic Process Automation, 机器人流程自动化) 的用武之地。

RPA 的本质：用软件模拟人的操作——打开浏览器、点击按钮、填写表单、下载文件——然后把这个过程录制下来，自动重复执行。

用 Codex / Claude Code 写 RPA 脚本

过去，写 RPA 脚本需要专业的开发人员。2026年，这件事已经可以用 Coding Agent 来做。

实际流程：

1. 把你要自动化的操作描述清楚：目标网站或系统、操作步骤、需要抓取或下载的内容、存储位置
2. 把这些描述输入 Codex 或 Claude Code，让它生成脚本
3. 在测试环境里运行，检查结果，告诉 Coding Agent 哪里不对
4. 迭代修改，直到脚本稳定运行

以税务局发票数据为例，你给 Codex 的描述大概是这样的：

帮我写一个 Python 脚本，实现以下操作：

1. 打开[税务局网站]，用账号密码登录
2. 进入“发票查询”页面，按月份筛选上个月的数据
3. 下载查询结果为 Excel 文件
4. 把文件重命名为“发票数据_YYYYMM.xlsx”，保存到指定路径
5. 如果下载失败，发送一条错误通知

登录信息和路径通过配置文件读取，不要硬编码在脚本里。

这个描述，Codex 能给你一个可以运行的初版脚本。你在测试中发现问题，继续在对话里告诉它，迭代修改。

重要提醒：脚本里涉及账号密码的部分，不要直接写进代码——通过配置文件或环境变量管理。这是基本的安全实践。

两种 RPA 路径

从实际操作经验来看，RPA 脚本有两条路：

API 路径（优先选择）：如果目标系统有 API，通过 API 取数是最稳定的方式。系统界面变了不影响脚本，速度快，错误处理也更干净。

网页巡检路径（系统没有 API 时的替代方案）：用脚本模拟浏览器操作。这条路有一个固有风险：目标网站的界面一旦更新，脚本可能就失效了，需要维护。把它理解成“过渡方案”，等目标系统开放 API 之后，迁移过去。

第二步：用 MCP 把 RPA 变成 AI 可调用的工具

RPA 脚本跑通之后，它还只是一个“定时自动执行的程序”——员工需要手动触发，或者等定时任务自动运行。

下一步，是把这些脚本封装成 AI 可以调用的工具。

这就是 MCP（Model Context Protocol）的作用。

MCP 是什么：一个开放协议，让 AI 模型可以标准化地调用外部工具和数据源。把你的 RPA 脚本封装成 MCP 服务之后，AI Agent 就可以像调用一个函数一样，触发这个脚本的执行。

封装之后发生了什么：

员工不再需要知道脚本在哪里、怎么运行。他们只需要告诉 AI：“把上个月发票数据下载下来放到公盘”——AI 调用 MCP 服务，触发脚本，脚本执行，文件出现在公盘。

从“员工手动操作系统”，变成了“员工向 AI 下达指令，AI 调用工具完成操作”。

除了 RPA 封装，还可以封装什么：

- 财务信息查询：把财务系统里常用的查询封装成 MCP 服务。员工问“这个供应商今年的应付账款余额是多少”，AI 直接查系统返回结果，而不是让员工自己登录系统查
- 数仓取数：把常用的 SQL 查询封装好，按参数触发，结果直接返回给 AI
- 文件操作：从公盘读取、写入、整理文件，成为 AI workflow 里的标准动作

第三步：双入口设计

工具封装好之后，需要考虑一个实际问题：不同成熟度的员工，怎么用这套工具？

有的员工习惯用 AI，有的员工不习惯，有的员工处于过渡阶段。如果只有一个入口——比如只能通过 Agent 调用——不习惯的员工就用不起来。

解决方案是双入口设计：

入口一：网页界面（人类友好入口）

做一个简单的网页界面，把常用操作做成按钮或表单。员工点击“导出上月报表”，后台触发对应的工具，结果送到公盘。

这个入口，完全不需要员工了解 AI 或 Agent，只要会点击就能用。

入口二：Agent 入口（AI 原生入口）

通过 Agent，员工用自然语言下达指令，Agent 判断意图，调用对应的 MCP 工具完成任务。

这个入口，适合已经习惯和 AI 交互的员工，或者需要组合多个操作的复杂指令。

两个入口的关键：它们调用的是同一套工具，只是触发方式不同。基础设施只需要建一次，两个入口共享。

这个设计解决了一个组织转型中的真实矛盾：你不能等所有人都准备好了再开始，但你也不能强迫所有人同时切换。双入口让不同进度的人都能用起来，而不是只有“先进”的那部分人在受益。

RPA 的三个原则：不只是点得快

很多人把 RPA 理解成“让机器点得更快”。

在简单场景里这没有问题。但在财务这个高风险场景里，自动化最重要的不是速度，而是可信。

一个可信的 RPA，需要满足三条原则：

原则一：要记录

每一个自动化动作，都要留下证据。

脚本运行了，下载了什么文件，存到了哪里，有没有报错——这些都要有日志记录。最好还有关键步骤的截图或状态变化记录。

原因很简单：财务场景出了问题，你需要能说清楚“脚本在这一步做了什么”，而不是“我以为它跑成功了”。

原则二：要暂停

不是所有步骤都适合一口气跑到底。

涉及提交、不可逆操作的节点，脚本应该在这里停下来，等人确认之后再继续。这不是因为 AI 不可信，而是因为这类操作本身就需要人的最终授权——自动化不改变这个事实。

原则三：要承认边界

RPA 能做的是：按固定步骤操作界面、下载文件、填写表单。

RPA 做不到的是：理解操作背后的业务逻辑、判断结果是否合理、在系统报错时知道该怎么办。

能点按钮，不等于理解后台业务规则。这个边界要在脚本设计阶段就明确——哪些情况应该停下来让人处理，而不是靠脚本自己猜。

一句话总结：好的 RPA 是“带证据的重复劳动接力”——机器做重复动作，人保留关键判断。

整条路的全貌

把三个步骤放在一起，这条路的全貌是：

手工重复操作
↓ 识别“不需要人判断的任务”
RPA 脚本（用 Codex / Claude Code 生成）
↓ 稳定运行后封装
MCP 工具（AI 可以调用的标准化接口）
↓ 部署双入口
网页入口 + Agent 入口
↓ 员工使用
人只负责下指令和验证结果

走完这条路，原来需要人手动做的那些重复操作，变成了基础设施——可以随时被调用，不依赖某个人记得去做，也不需要某个人守在那里等它跑完。

一个需要说清楚的期望管理

这条路不是一蹴而就的。

RPA 脚本需要维护。目标系统更新了界面，脚本可能失效。特别是网页巡检类的脚本，维护成本比 API 调用高得多。把维护时间算进去，才是真实的投入产出。

MCP 封装需要一定技术基础。不是说员工自己做不到，但需要有人（可能是你自己，可能是技术同事，可能是 Coding Agent）来负责封装和维护这层。

双入口的网页界面需要开发。好消息是，Codex 可以帮你做这件事，不需要专业前端开发人员。坏消息是，它仍然需要时间和迭代。

把这些成本如实估进去，这条路仍然是值得走的——因为一旦基础设施搭好，它带来的效率提升是持续的，不是一次性的。

□ 本章要点卡

1. 识别待封装能力的标准：不需要人判断、只需要人执行的任务——这类任务都是候选
2. 三步路径：RPA固化 → MCP封装 → 双入口部署——每一步都有独立价值，不需要一次全部到位
3. 双入口解决真实矛盾：不同成熟度的员工可以共存，基础设施只建一次，两个入口共享

□ 读完这章，你可以做的事

- [] 列出你们财务部门每月重复频率最高的三个手工操作，对照“不需要人判断”这个标准，哪些是候选？
- [] 选最简单的那个，试着用 Codex 或 Claude Code 写一个初版脚本——不需要完美，能跑起来就算成功
- [] 想想你们团队里最不习惯用 AI 的那个人——如果要让他也能用上这套工具，网页入口应该长什么样？

下一章：AI 提问，人澄清，数据说话——财务分析的人机分工框架

第三章：AI 提问，人澄清，数据说话

读完这章，你会有一个清晰的财务分析人机分工框架——知道哪些交给 AI，哪些必须由人来做。

一个看起来很简单的问题

“这个月的差旅费为什么增加了？”

这个问题，财务人员每个月都会被问到，或者自己也会问。

你可能觉得：给 AI 这个问题，它应该能回答。毕竟数据都在那里。

但事实是：AI 能告诉你差旅费增加了多少、哪个部门增加最多、和去年同期相比增幅是多少——但它没有办法告诉你为什么。

“为什么”的答案，在数据之外。它在项目经理的脑子里，在业务部门刚刚做的那个决定里，在上个月临时批了一批出差申请的那封邮件里。

AI 看得见数字，看不见业务现实。

这个洞察，是整个财务分析人机分工框架的起点。

财务分析的两个层次

财务分析，从本质上可以分成两个层次：

数据层： 数字是什么，变化了多少，和什么对比，趋势是怎样的，ROI 是多少，偏差有多大。

业务层： 为什么是这个数字，数字背后发生了什么，这个趋势意味着什么，应该怎么应对。

AI 在数据层，表现非常可靠。

计算、对比、趋势识别、异常标注、ROI 测算——这些都是结构化的、可以精确描述的任务。配有代码执行能力的 AI，在这些任务上准确率高，速度快，而且不会

因为工作量大而出错。

AI 在业务层，无法替代人。

因为业务层的答案不在数据里。它需要和业务部门沟通，需要了解组织内部正在发生的事情，需要判断哪个解释是真实原因而不只是表面相关。

这两个层次的分工，就是这一章的核心框架。

框架：AI 提问，人澄清，数据说话

把这个分工具体化，是这样一个三步循环：

第一步：数据说话

AI 处理数据，生成结构化的分析结果。

这一步，AI 做的事情包括：

- 按维度汇总数据（部门、项目、科目、时间段）
- 计算关键指标（增减额、增减率、占比、ROI）
- 和基准对比（预算 vs 实际、同比、环比）
- 标注异常（超预算、偏离趋势、数据缺失）
- 生成可视化报表（图表、穿透表格）

输出的结果，是一份结构清晰的数据摘要——数字准确，维度完整，异常已经标注出来。

关键原则：这一步的输出，只描述“是什么”，不解释“为什么”。AI 不要在这一步给出业务解读，因为它没有足够的信息做出可靠的解读。

第二步：AI 提问

基于数据层的分析，AI 生成一份“需要澄清的问题清单”。

这是这个框架里最有价值、也最容易被低估的一步。

AI 的问题，来自它观察到的数据异常和模式：

- “差旅费同比增加38%，主要集中在华南区域。是否有特定项目导致了这个增加？”
- “研发费用在Q3出现了一次性支出，金额是全年预算的21%。这笔支出的业务背景是什么？”
- “应收账款周转天数从45天增加到67天。是回款政策变化，还是特定客户的付款延迟？”

这些问题，AI 自己回答不了——但它能把“值得去问的问题”系统性地列出来，而不是让财务人员凭经验去猜哪里有问题。

一个提示词框架，用于生成问题清单：

基于以下数据分析结果，请生成一份需要向业务部门澄清的问题清单。

要求：

- 只提出数据本身无法解释的问题
- 每个问题要说明：观察到什么现象、为什么需要澄清
- 按优先级排序（对财务结果影响最大的问题排在前面）
- 不要自己猜测答案，只提问

数据分析结果：[粘贴AI生成的数据摘要]

第三步：人澄清

财务人员带着 AI 生成的问题清单，去和业务部门沟通。

这一步，是人不可替代的地方：

判断哪个问题最重要。 AI 按数据影响大小排了优先级，但你知道哪个问题在政治上敏感，哪个部门负责人最好在什么时机沟通，哪个问题可能引发更大的讨论。

获取数据之外的信息。 业务部门的回答，往往不是一个数字，而是一段背景——“因为我们在Q3拿下了一个新项目，差旅是为了客户拜访”。这类信息，AI 没有办法自己获取。

做出判断。 听完业务部门的解释，你来判断：这个解释合理吗？需要进一步核实吗？这对财务预测有什么影响？

澄清完成之后，把业务部门的回答输入给 AI，让它更新分析结论，生成最终的财务解读。

可视化报表：让数据说话的载体

这个框架需要一个展示数据的载体，让“数据说话”这一步的结果，能被清晰地看见和穿透。

这就是可视化管理报表的作用。

一个好的财务可视化报表，需要满足三个要求：

第一：看得见异常。不只是展示数字，而是把异常自动标注出来——超预算的用红色，同比大幅波动的用标注，这样财务人员不需要逐行扫描，能直接看到哪里值得关注。

第二：能穿透到底层数据。看到汇总数字有异常，可以点击下钻，看到是哪个部门、哪个项目、哪笔具体支出导致的。这个穿透能力，是把“数据层”和“业务层”连接起来的桥梁。

第三：AI 框架性分析基于事实。报表里的 AI 解读，约束它只能基于数据事实——“差旅费增加38%，主要来自华南区域”，而不是“差旅费增加可能是因为……”。业务解读，留给人去做。

关于工具：用 Codex 可以开发这类可视化报表——给它你的数据结构和展示需求，它能生成可以直接部署的前端应用。不需要专业前端开发，但需要你能清楚地描述你想要什么。

这个框架的边界

诚实地说，这个框架有它适用的范围，也有它不适用的地方。

适用的场景：

- 定期的管理报告（月度、季度财务回顾）
- 预算 vs 实际的差异分析

- 成本结构分析
- ROI 测算和对比
- 异常检测和预警

不适用的场景：

- 需要主观判断的估值（企业并购估值、资产减值测试）
- 涉及会计政策选择的判断（这类判断有法律责任，不能外包给 AI）
- 需要行业专业知识的分析（AI 的行业理解停留在通用层面）
- 高度政治敏感的财务决策（谁来承担成本超支的责任）

判断标准很简单：如果你不敢在审计师面前说“这个结论是 AI 给的”，那就不要让 AI 做这个结论。

一个完整的月度财务回顾流程

把这一章的所有内容放在一起，一个 AI 辅助的月度财务回顾可以这样运作：

第1-2天：数据准备（AI + RPA） 自动从财务系统导出数据（参考第二章的基础设施），AI 做初步清洗和结构化处理。

第3天：数据分析（AI） 按维度汇总，计算关键指标，标注异常，生成可视化报表，输出问题清单。

第4-5天：业务澄清（人） 财务人员带着问题清单，和各业务部门沟通，收集业务背景和解释。

第6天：分析结论（人 + AI） 把业务澄清的结果输入给 AI，AI 更新数据解读，人做最终的判断和措辞，生成报告初稿。

第7天：审核和发布（人） 财务负责人审核报告，确认数字准确、解读合理、措辞得当，发布。

这个流程，不是在用 AI 替代财务人员，而是在让财务人员把时间花在真正需要人的地方——业务沟通和最终判断，而不是数据处理和报表格式。

□ 本章要点卡

1. 财务分析有两个层次：数据层（AI可靠）和业务层（人不可替代）——这个分工，是所有财务AI应用的基础
 2. 三步循环：数据说话 → AI提问 → 人澄清——AI做的最有价值的事之一，是生成“值得去问的问题”
 3. 约束AI只说事实：报表里的AI解读只描述“是什么”，“为什么”留给人和业务部门去回答
-

□ 读完这章，你可以做的事

- [] 找一份你最近做的差异分析，试着把数据给 AI，让它生成一份问题清单——看看它问的问题，有没有你自己没想到的角度
 - [] 检查你们现在的管理报表：AI 的解读有没有在做业务猜测？如果有，加一条约束：“只描述数据事实，不解释业务原因”
 - [] 想一想你下个月的财务回顾：哪些环节可以用这一章的框架来重新设计？
-

下一章：财务 AI 的红线——比通用版更严格的边界

第四章：自动化之前，先让流程可见

读完这章，你会明白为什么“先把流程写下来”，是所有自动化的真正第一步。

一个很常见的误解

一提到 AI 和自动化，很多人的第一反应是：赶快写代码，让它跑起来。

这个直觉可以理解——AI 现在确实能很快生成脚本、搭建工作流、部署工具。但在财务场景里，这个顺序往往会带来麻烦。

因为有一件事被跳过了：把流程从脑子里拿出来。

流程在哪里？

财务部门的很多流程，不存在于任何文档里。

它存在于老员工的脑子里，存在于“我们一直这么做”的习惯里，存在于聊天记录和零散的邮件附件里。

这会带来几个实际问题：

换人就断。某个人离职，那个流程就不知道该怎么走了，或者走错了也没人发现。

交接靠口头。新人来了，旁边的人讲一遍，讲的人记得多少，新人吸收多少，完全不可控。

异常处理不透明。正常情况大家都会，但遇到例外，每个人处理方式不一样，结果也不一致。

Agent 无法稳定接手。这是和 AI 最直接相关的问题——如果流程说不清楚，Agent 就没有办法可靠地执行它。你没有办法把一个“靠经验判断”的流程交给 AI 自动化。

让流程可见的三步

第一步：识别流程

从现有的材料里，把流程的骨架找出来。

材料来源可以包括：

- 现有的 SOP 文档（哪怕已经过时）
- 聊天记录里反复出现的操作步骤
- 老员工的口头描述（录音后转文字，交给 AI 整理）
- 历史的错误记录和纠错过程

这一步不需要完美，先把流程的主干拿出来，再补细节。

AI 可以帮什么：把杂乱的文字描述，整理成有结构的步骤列表。你给它聊天记录或录音文字，它帮你提取出“步骤1、步骤2、步骤3”，节省大量整理时间。

第二步：画成结构

有了骨架之后，把流程落到图上。

一个完整的流程图需要包含：

角色：这个步骤由谁来做——财务、业务部门、系统自动完成、还是外部供应商。

节点：每个步骤做什么，产生什么输出。

分支：遇到不同情况时走哪条路——正常情况怎么走，异常情况怎么走。

审批点：哪些步骤需要人审批才能继续。

输出物：每个步骤完成后，产出的是什么——报表、凭证、邮件、还是系统里的一条记录。

这张图，不需要特别专业的工具。用流程图软件（draw.io、飞书文档的流程图功能）甚至白板都可以。

关键不是画得漂亮，而是把决策节点和例外情况都画出来。

第三步：用图说话

流程图画出来之后，它的价值才真正开始体现。

让流程可以被讨论。把图放出来，让相关的人一起看——财务、业务、IT。每个人看图的时候，都会发现“这里不对”或者“这里我们实际上是这样做的”。这些修正，会让流程越来越接近真实。

让流程可以被复用。下次有新人，给他看这张图，而不是靠口头传授。

让流程可以被改进。当你想优化某个环节，图上一目了然——哪个节点最慢、哪个分支最容易出错、哪个审批最冗余。

让流程可以被 Agent 执行。这是和 AI 最直接的连接。

流程可见之后，Agent 才能接手

一个流程，如果能被人用语言清楚地描述，通常就能被 Agent 执行。

Agent 执行流程的条件：

- 步骤是确定的，不依赖隐性经验
- 每个节点的输入和输出是明确的
- 分支条件是可以判断的（不是“靠感觉”）
- 例外情况有明确的升级路径

如果流程图里有“视情况而定”、“一般来说”、“老员工知道怎么处理”——这些地方，Agent 就没有办法可靠地执行，需要人介入。

不是所有流程节点都适合自动化。把哪些节点交给 Agent、哪些节点留给人，这个判断在流程图上做，比在代码里做要容易得多。

一个真实的例子：财务审批流程的可见化

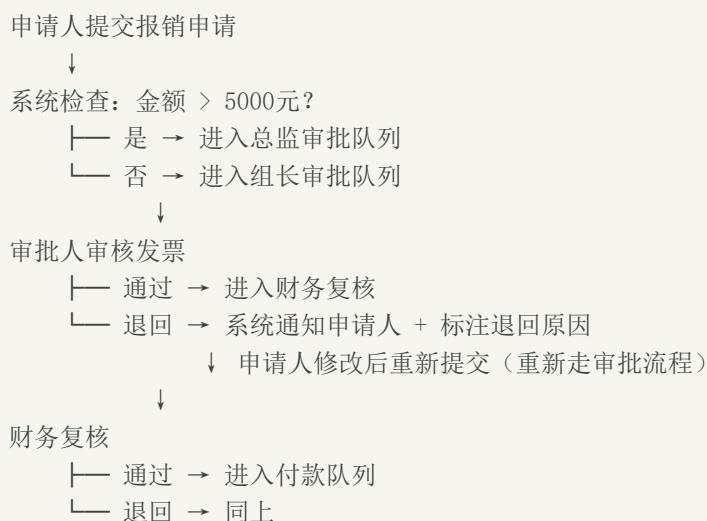
以一个常见的费用报销审批为例。

流程可见之前的状态：

“报销就是提交申请，然后等审批。金额大的要总监签，小的组长签就行。有时候发票有问题会打回来，重新提交。”

这段描述，有信息，但没有结构。什么叫“金额大”？发票哪里有问题？打回来之后谁来通知申请人？重新提交走什么流程？

流程可见之后：



这个流程图，让以下事情变成可能：

- Agent 可以自动判断审批路由
- 系统可以自动发送通知
- 退回原因可以标准化
- 流程状态可以实时追踪

流程可见的顺序不能颠倒

自动化之前，先标准化。 标准化之前，先让流程可见。

这个顺序不能颠倒。

如果流程还说不清楚就开始自动化，结果是把混乱执行得更快。 错误发生的速度更快，范围更大，而且更难追查——因为“系统自动做的”比“人做的”更难说清楚出了什么问题。

这是很多 RPA 和 AI 自动化项目失败的真实原因——不是技术问题，是流程没有想清楚就开始做了。

让流程可见，是真正有效的自动化的前提，不是可选项。

☐ 本章要点卡

1. 财务流程大量存在于人的经验里——流程不可见，自动化就不可靠
 2. 让流程可见的三步：识别骨架 → 画成结构 → 用图说话——顺序不能颠倒
 3. 流程图里的“视情况而定”，就是人必须介入的节点——在图上决定，比在代码里决定容易得多
-

☐ 读完这章，你可以做的事

- [] 选你们部门一个“全靠经验”的流程，花30分钟，试着用文字把每一个步骤写出来——写不清楚的地方，就是流程还没想清楚的地方
 - [] 把上面的文字描述交给 AI，让它生成一个结构化的流程步骤列表，看看它整理出来的和你脑子里的有没有差距
 - [] 找一个流程图工具，把这个流程画出来——特别注意标出分支条件和审批节点
-

下一章：财务 AI 的红线——比通用版更严格的边界

第五章：财务 AI 的红线

读完这章，你会知道财务场景里哪些事绝对不能让 AI 做，以及为什么比通用版的边界更严格。

为什么财务需要单独的红线清单

Part I 的第三章和第四章，已经讲过 AI 的通用局限和数据安全边界。

财务场景需要在这个基础上，再加一层——因为财务工作有两个特质，使得“出错”的代价比其他职能高得多：

数字有法律效力。财务报表、税务申报、合同金额、监管报告——这些文件里的数字，一旦错了，后果不只是“数据不准”，而是审计风险、税务罚款、法律责任。

错误难以撤回。一封邮件发错了可以道歉，一个报告发错了可以更正，但一份已经提交给监管机构的申报、一笔已经完成的付款、一份已经签署的合同——这些操作的可逆性极低。

这两个特质，决定了财务场景的 AI 使用边界，必须比通用版更保守、更明确。

红线一：财务数字不能只靠 AI 生成

AI 可以帮你计算、汇总、对比——但任何出现在正式文件里的财务数字，必须经过人工核实。

不是“AI 算完人看一眼”，而是独立验算。

看一眼和验算，是两件不同的事。

看一眼：数字看起来合理，感觉没问题，通过。

独立验算：用原始数据，按照自己的计算逻辑，重新算一遍，和 AI 的结果对比，找出差异原因。

在财务报表、税务申报、对外披露文件里，所有关键数字，都需要独立验算。这不是对 AI 不信任，而是财务工作本来就有的双重复核要求——只是现在复核的对象

从“同事的计算”变成了“AI 的输出”。

红线二：会计判断不能外包给 AI

会计判断，是指需要运用会计准则、职业判断和主观估计的决定。比如：

- 某项资产是否需要计提减值，减值金额是多少
- 某笔收入应该在哪个时间点确认
- 某类费用应该资本化还是费用化
- 关联交易的定价是否公允

这类判断，AI 可以给你参考意见，可以帮你列出相关准则条款，可以帮你整理历史先例——但最终判断必须由有资质的财务专业人员来做，并且承担责任。

原因不只是技术——AI 确实对会计准则有一定了解。更根本的原因是：这类判断有法律效力，需要一个能够承担法律责任的人来做出。AI 不能承担责任，所以它不能做这类判断。

如果你不敢在审计师面前说“这个判断是 AI 做的”，那就不要让 AI 做这个判断。

红线三：税务申报内容不能直接用 AI 生成

税务申报是高风险的对外文件，错误可能触发税务稽查、罚款、甚至刑事责任。

AI 可以帮你：

- 整理申报所需的数据
- 检查申报数据的内部一致性
- 解释税法条款的一般含义
- 提醒可能遗漏的申报项目

AI 不能帮你：

- 直接生成申报数据

- 解释税法条款在你的具体情况下如何适用
- 决定某项收入或支出的税务处理方式

税务判断需要了解你的具体业务背景、当地税务机关的执行口径、以及最新的政策变化。这些信息，AI 不一定有，也不一定准确。

红线四：付款和资金操作不能由 AI 自动触发

这是最严格的一条红线，没有例外。

任何涉及真实资金流动的操作——付款审批、转账指令、资金划拨——必须由有授权的人来最终触发，不能由 AI 或自动化脚本直接完成。

这不只是内控要求，也是基本的风险防范。一旦 AI 或 RPA 出现错误，或者被恶意利用，能够自动触发付款的系统，造成的损失可能是不可逆的。

付款环节，人工确认是硬性要求，不是可以用效率理由绕过的。

红线五：高敏感财务数据不能进入公共 AI

Part I 已经讲过数据安全的通用原则。财务数据的敏感性，需要额外强调。

以下数据，只能在企业批准的、有数据保护协议的 AI 渠道里处理：

- 未公开的财务报表数据
- 预算和预测数据
- 并购、融资、上市相关的财务信息
- 客户的财务数据
- 薪酬和个人所得税数据

这些数据一旦通过个人账号进入公共 AI，你对这些数据的控制就已经失去了。即使 AI 平台承诺不存储，处理过程本身已经发生在企业边界之外。

红线六：AI 的财务解读不能不加审核直接对外

AI 生成的财务分析文字，在对外发布之前，必须经过专业财务人员的审核。

原因有两个：

准确性风险： AI 可能在解读中引入错误——数字引用错误、逻辑推断偏差、对业务背景的误解。

措辞风险： 财务对外沟通有非常严格的措辞要求，特别是上市公司——某些表述可能触发信息披露义务，某些措辞可能被解读为业绩承诺或风险隐瞒。AI 不了解这些合规边界。

财务文字的审核，不只是看数字对不对，还要看措辞是否合规、是否可能引发误解。

一张快速判断表

遇到一个具体任务，用这张表快速判断：

任务类型	AI 可以做	AI 不能做
数据汇总和计算	生成初稿	最终数字不能不验算
会计准则查询	列出相关条款	不能做具体情况的适用判断
税务数据整理	整理和检查	不能生成申报内容
付款流程	辅助审核材料	不能触发付款操作
财务分析文字	生成初稿	不能不审核直接对外
敏感数据处理	企业版渠道	个人账号/公共AI
异常标注	识别和提示	不能自行判断处理方式

红线的目的不是阻止使用

说了这么多“不能”，这一章的目的不是让你对财务 AI 望而却步。

财务 AI 能带来的效率提升是真实的。这些红线，是为了让你在安全的边界内，放心地去探索和使用——而不是等出了问题再来划线。

知道边界在哪里，才能在边界内自由探索。

☐ 本章要点卡

1. 财务数字的法律效力和低可逆性，决定了财务场景的红线比通用版更严格
2. 六条红线：数字独立验算、会计判断不外包、税务申报不直接生成、付款人工触发、敏感数据只走企业渠道、对外文字必须审核
3. 红线的目的是划定安全空间，不是阻止使用——知道边界，才能在边界内放心探索

☐ 读完这章，你可以做的事

- [] 对照六条红线，检查你们团队现在的 AI 使用方式——哪条红线目前没有明确规定？
- [] 找到你们日常工作里最接近红线的一个场景，想想目前的做法是否足够安全
- [] 如果你管团队，考虑把这六条红线写成一页纸的财务AI使用规范，在团队里明确

下一章：三个真实场景——从想法到可用工具的完整过程

第六章：三个真实场景

这一章不讲原则，只讲过程——三个从真实需求出发，一步一步做出来的财务 AI 工具。

为什么讲过程，不讲结论

很多 AI 案例的写法是：问题是什么，解决方案是什么，效果是什么。

这种写法看起来整洁，但没有告诉你最重要的东西：中间经历了什么，哪里卡住了，怎么做决定的。

这一章反过来——重点写过程，写决策，写踩过的坑。结论反而放在最后，因为结论只有在你理解了过程之后才有意义。

场景一：费用汇总对账工具

需求的起点

需求听起来很朴素：上传费用明细表和规则表，按口径匹配对象，输出一张汇总表。

像是 Excel 里的 vlookup，不高级。但它有一个好处：足够具体，能验证。

第一个决定：做网页，不做命令行脚本。

原因很现实——非技术同事不会配置环境，不一定会跑命令行，但可以打开网页、上传文件、看结果。工具如果用不起来，再精巧也没用。

用 Codex 生成前端页面，这个决定省了大量来回。

六个阶段，一步一步长出来

第一阶段：先跑通

上传两张表，按成本中心匹配，输出第一张汇总。

这一步的重点不是复杂，而是第一张表能对上。如果第一张表就对不上，后面加任何功能都是在错误的基础上叠加错误。

跑通了，再往下走。

第二阶段：数据变了怎么办

真实工作里，表头会变，字段位置会变，规则表会更新。不能假设每个月文件都一样。

这时要让 AI 重新理解字段，用样例跑效果，确认读取逻辑在新的文件格式下仍然成立。

这一步踩过的坑：“能运行”不等于“读对了”。脚本跑完没有报错，但字段读取偏移了一列，结果全错。这种错误如果不仔细对账，很难发现。

第三阶段：对账排查

当系统结果和人工 Excel 不一致时，正确的做法不是只说“你错了，重新算”，而是要求逐项检查：

- 字段读取是否正确
- 匹配规则是否按预期执行
- 空值如何处理
- 汇总口径是否一致
- 差异的具体来源是哪里

让 AI 把每一步的中间结果展示出来，而不是只给最终结果。这样当结果不对的时候，能找到问题在哪一步。

关键认知：业务同学要结果，审计要过程。工具不能只给结果，还要让人能复核计算过程。这个设计要在早期就考虑进去，不是事后加。

第四阶段：注入业务逻辑

简单匹配跑通之后，真实的业务规则开始进入工具：

- 某些科目需要合并处理

- 共享费用按比例分摊给多个主体
- 分摊比例每个季度可能调整
- 跨币种的汇率换算
- 手工调整项的处理

每一条业务规则，都需要先把逻辑说清楚，再让 AI 实现。说不清楚的规则，AI 会猜，猜的结果大概率不对。

第五阶段：规则配置化

如果规则下个月还会改，就不要每次改代码——规则应该做成可读、可调整的配置页。

财务同事可以在界面上直接修改分摊比例、调整匹配规则，不需要找人改代码，不需要等待技术支持。

这一步的投入，会在后续每次规则变化时节省大量时间。

第六阶段：产品化

最后，才是让工具看起来像一个正常产品：

- 数字显示千分位和两位小数
- 表格有合计行和正式标题
- 去掉开发阶段的调试信息
- 导出的文件名自动包含日期

这些细节不影响计算逻辑，但影响使用体验——一个看起来“粗糙”的工具，会让人对结果的信任度降低，哪怕计算完全正确。

这个案例说明了什么

工具不是一次写完的，而是在对账和反馈中逐步长出来的。

先跑通，再对账，再加业务逻辑，再配置化，最后产品化。这个顺序不能颠倒——在没有跑通之前加复杂业务逻辑，是在错误的地基上盖楼。

场景二：财务规则查询助手

需求的起点

财务规则常常分散在很多地方：文档、表格、历史邮件、老员工的记忆、临时通知。

员工遇到问题，不知道问谁。即使问到了，不同的人在不同时间给出的答案，口径可能不一致。

这个场景里最大的风险不是没人答，而是每个人答得都像真的。

关键设计决策

这个工具的目标，不是让 AI 当财务专家，而是做一个统一查询入口：

- 查公司主体信息
- 查项目代码
- 查发票报销规则
- 查常见问题的标准口径
- 遇到不确定的问题，走确认路线

最重要的设计原则：知道什么时候不能回答。

AI 很容易给出一个看起来完整的答案，即使它不确定。但在财务场景里，一个“完整但错误”的答案，比“我不确定，请确认”危害大得多。

所以这个工具有一条硬性规则：对于不在知识库里的内容，或者涉及具体业务判断的问题，工具必须明确说“这个问题需要人工确认”，而不是猜测。

这条规则，要在工具的提示词里硬编码——不能靠模型“懂事”，要靠设计来保证。

从“问人”到“问入口”的转变

工具上线之前的状态：

- 员工到处问，找不到负责人
- 找到了，答案也可能过时
- 不同人给的答案口径不同

工具上线之后：

- 先查统一入口
- 能回答的，给标准口径和来源文档
- 不能回答的，标注“请联系财务部确认”，并告诉员工找谁

更重要的变化是：系统把“不确定”变成了流程的一部分，而不是让它在口口相传中消失。

踩过的坑

坑一：知识库不更新，工具就失效。

规则变了，工具里的内容还是旧的——员工得到旧答案，按旧规则操作，出了问题还以为是按规定来的。

知识库需要有人负责维护，这是工具能否持续可信的关键，和技术无关。

坑二：员工对 AI 的信任度比对人更高。

反直觉但真实——很多员工对 AI 给的答案，反而比对同事给的答案更容易接受，因为“系统说的”感觉更权威。

这意味着：工具给出错误答案的风险，比人给出错误答案的风险更大，因为传播速度更快，质疑的人更少。

这是为什么“知道什么时候不能回答”这个设计，比“尽量回答更多问题”更重要。

场景三：月度财务回顾的 AI 辅助流程

需求的起点

月度财务回顾，是财务部门最重复、也最重要的工作之一。

重复的部分：数据收集、汇总、生成图表、写差异说明。 重要的部分：业务解读、异常判断、和管理层的沟通。

问题是：大量时间花在了重复的部分，留给重要部分的时间反而不够。

重新设计之后的流程

第1-2天：数据自动到位

RPA 从财务系统自动导出月度数据，存到公盘指定位置。不需要人手动登录系统操作，不需要等某个人有空来做这件事。

（这是第二章讲的基础设施发挥价值的地方。）

第3天：AI 做数据层分析

AI 读取数据，生成：

- 按维度汇总（部门、项目、科目）
- 关键指标计算（预算完成率、同比、环比）
- 异常标注（超预算项目、大幅波动）
- 可视化图表
- 需要业务澄清的问题清单

这一步，AI 只描述“是什么”，不解释“为什么”。

第4-5天：财务人员做业务澄清

带着 AI 生成的问题清单，和各业务部门沟通，收集业务背景。

这是人不可替代的部分。AI 看得见数字，看不见业务现实——为什么差旅费增加、某个项目为什么延期、某笔支出的真实业务背景是什么，这些答案在业务部门，不在数据里。

第6天：人和 AI 共同完成分析结论

把业务澄清的结果输入给 AI，AI 更新数据解读，财务人员做最终判断和措辞确认。

这里有一个关键实践：约束 AI 的分析只能基于已知事实，不能做超出数据的推断。

提示词框架：

以下是本月财务数据和业务部门的说明。
请基于这些已知信息，生成月度回顾的分析文字。

要求：

- 只描述数据事实和业务部门已确认的原因
- 不要推测业务部门没有确认的内容
- 对于原因不明的异常，标注“原因待确认”
- 语气客观，不做价值判断

第7天：审核和发布

财务负责人审核报告，确认数字准确、解读合理、措辞符合对外沟通要求，发布。

这一步不能省——这是财务负责人对整份报告质量背书的环节。

这个流程改变了什么

改变之前：大量时间在数据收集和格式整理，业务分析的时间不够。

改变之后：数据收集和格式整理几乎全部自动化，财务人员的时间集中在业务澄清和判断上——这才是财务工作真正的价值所在。

不是用 AI 替代财务人员，而是让财务人员做更有价值的事。

三个案例的共同主线

回顾这三个场景，有一条共同的主线：

先把想法做成能用的东西，在使用和反馈中把它做好。

费用汇总工具，先跑通，再对账，再加逻辑，再产品化。 财务查询助手，先建知识库，再设计边界，再上线，再根据反馈更新。 月度回顾流程，先跑通一个月，再根据实际使用调整节奏和工具配置。

没有一个是“想清楚了再做”——因为很多东西，不做就想不清楚。

这不是草率，这是有意识地选择“用最小的成本，让判断发生”。

☐ 本章要点卡

1. 工具是在对账和反馈中长出来的——先跑通，再对账，再加逻辑，再产品化，这个顺序不能颠倒
2. 可信查询系统的关键能力：知道什么时候不能回答，比“尽量多回答”更重要
3. AI 辅助流程改变的不是工作的性质，而是时间分配——重复性工作自动化，让人专注于判断和沟通

☐ 读完这章，你可以做的事

- [] 对照费用汇总工具的六个阶段，找出你们团队里一个类似的重复性数据处理任务——它现在处于哪个阶段？
- [] 想想你们是否有财务规则分散、口径不一致的问题——一个查询助手能解决的最核心的三个问题是什么？
- [] 下个月的财务回顾，试着用这一章的流程框架重新安排一下时间分配——数据收集和AI分析放在前几天，业务澄清时间保留够

写在 Part III 结尾

这本 Finance AI Playbook，不是一份技术手册，也不是一份方法论清单。

它是一个财务人走过这条路之后，把走过的路记下来的东西。

有些地方还在摸索，有些工具还在迭代，有些判断可能在六个月后需要更新。但走过这段路最重要的收获是：

AI 在财务场景里能做的事，比大多数人想象的多。但它做不到的那些事——判断、沟通、承担责任——恰恰是财务工作最核心的价值所在。

把重复的交给工具，把判断留给人。

这不是退让，这是财务专业的正确归宿。

全书完
